

1. Objetivo del Proyecto

Diseñar y desarrollar una aplicación robusta para la gestión de inventario de múltiples sucursales dentro de una misma organización. El proyecto no se evalúa únicamente por su funcionamiento, sino por la calidad del diseño, la solidez de la arquitectura, la claridad de la documentación y la incorporación inteligente de herramientas de inteligencia artificial durante el proceso de desarrollo.

*Principio rector: cada decisión de diseño debe poder responder claramente a la pregunta '**¿Por qué se hizo así?**'. La justificación técnica es tan importante como el resultado final.*

2. Descripción del Problema y Alcance

La solución debe permitir que varias sucursales de una organización gestionen su inventario de forma independiente, pero compartiendo visibilidad sobre el inventario general. Esto implica que cada sucursal opera con autonomía operativa mientras mantiene coherencia de datos con el resto de la red.

2.1. Características por Sucursal

- Opera de forma completamente independiente en sus transacciones locales.
- Comparte información de inventario con las demás sucursales en tiempo real o near-real-time.
- Puede consultar el inventario de cualquier otra sucursal de la red.
- Puede solicitar y recibir transferencias de producto entre nodos de la red.

3. Módulos Funcionales del Sistema

3.1. Gestión de Inventario (CRUD Completo)

El módulo de inventario constituye el núcleo del sistema. Debe permitir las operaciones básicas de creación, consulta, actualización y eliminación de registros de producto, con las siguientes capacidades específicas:

- Visualizar el catálogo de productos disponibles en la sucursal propia.

- Consultar el inventario de cualquier otra sucursal de la red.
- Registrar el ingreso de productos (compras, devoluciones, ajustes de inventario).
- Registrar el retiro de productos (ventas, mermas, ajustes de inventario).
- Controlar stock mínimo y generar alertas de reabastecimiento.
- Gestionar múltiples unidades de medida por producto.

Importante: El sistema debe mantener trazabilidad completa de los movimientos. Cada ingreso o retiro debe quedar registrado con fecha, responsable, motivo y cantidad, garantizando un historial auditable.

3.2. Módulo de Compras

El sistema debe soportar el ciclo completo de adquisición de productos:

- Crear y gestionar órdenes de compra a proveedores.
- Registrar condiciones de la compra: precio unitario, descuentos, plazo de pago.
- Actualizar automáticamente el inventario al confirmar la recepción de mercancía.
- Llevar histórico de compras por proveedor y por producto.
- Calcular el costo promedio ponderado del inventario.

3.3. Módulo de Ventas

Para el registro y control de las salidas comerciales de producto:

- Registrar transacciones de venta por producto, cantidad y precio.
- Asociar cada venta a una sucursal, fecha y responsable.
- Validar disponibilidad de stock antes de confirmar la venta.
- Aplicar descuentos y gestionar diferentes listas de precios.
- Generar comprobantes o registros de venta para consulta posterior.

3.4. Transferencia de Productos entre Sucursales

Este módulo gestiona el flujo completo de traslado de mercancía entre nodos de la red, cubriendo todo el ciclo desde la solicitud hasta la confirmación de recepción:

1. Solicitud de transferencia: la sucursal destino (o un administrador) genera una solicitud formal indicando producto, cantidad y origen.

2. Preparación del envío: la sucursal origen revisa disponibilidad y confirma o ajusta la cantidad a enviar.
3. Registro de envío: se registra el despacho con fecha estimada de llegada y transportista.
4. Confirmación de recepción completa: el inventario de la sucursal destino se actualiza automáticamente.
5. Confirmación de recepción parcial: se registra la diferencia (faltantes), se genera una alerta y se define el tratamiento (reenvío, ajuste o reclamación).

3.5. Módulo de Tiempos de Envío y Logística

Para el seguimiento del estado y duración de los traslados:

- Registrar y consultar tiempos estimados vs. tiempos reales de entrega.
- Clasificar rutas por prioridad, costo o tiempo.
- Visualizar el estado de cada transferencia en curso (en preparación, en tránsito, recibido, con faltantes).
- Generar reportes de cumplimiento logístico por sucursal y por ruta.

3.6. Análisis y Visualización (Dashboard)

Cada sucursal debe contar con herramientas de análisis que le permitan tomar decisiones informadas. El dashboard debe presentar, como mínimo:

- Volumen de ventas del mes en curso vs. meses anteriores.
- Comportamiento del inventario: rotación, productos de alta y baja demanda.
- Estado de las transferencias activas y su impacto en el inventario.
- Indicadores de reabastecimiento: productos próximos a agotarse.
- Comparativa de rendimiento entre sucursales (visible para perfiles administrativos).

La información puede presentarse mediante gráficas, dashboards interactivos o indicadores clave de desempeño (KPI). Se valora la claridad visual y la relevancia de la información presentada.

4. Funcionalidad Adicional Propuesta

Además de los módulos obligatorios, se espera que el candidato proponga e implemente al menos una funcionalidad adicional que aporte valor real al sistema. A continuación se presentan algunas ideas orientadoras (no limitantes):

Funcionalidad	Descripción	Valor agregado
Sistema de alertas inteligentes	Notificaciones automáticas cuando un producto supera o cae por debajo de umbrales configurables de stock. Puede incluir alertas por correo o en la interfaz.	Alto valor operativo
Predicción de demanda	Modelo básico (regresión lineal o promedio móvil) para estimar el consumo futuro de un producto con base en el historial de ventas.	Diferenciador técnico
Gestión de proveedores	Módulo para registrar proveedores, asociarlos a productos, registrar condiciones comerciales y evaluar tiempos de entrega.	Complemento natural
Control de caducidad	Para productos perecederos, seguimiento de fechas de vencimiento y alertas preventivas de descarte o promoción.	Aplicable a múltiples industrias
Auditoría y trazabilidad	Registro detallado de cada acción realizada sobre el inventario: quién lo hizo, cuándo y por qué.	Esencial para cumplimiento
Módulo de reportes exportables	Generación de reportes en PDF o Excel para movimientos de inventario, ventas o transferencias en un rango de fechas.	Alta utilidad práctica

5. Reglas Técnicas Obligatorias

La arquitectura de la solución debe cumplir con los siguientes requisitos técnicos sin excepción:

Requisito	Descripción
Separación de capas	La solución debe tener al menos tres capas separadas: frontend, backend y base de datos, cada una con responsabilidades claramente definidas.
Comunicación por API	El frontend debe comunicarse con el backend exclusivamente a través de una API bien definida (REST o GraphQL). No se acepta lógica de negocio en el cliente.

Contenedorización	El proyecto completo debe poder ejecutarse con un solo comando usando Docker Compose. No deben existir dependencias de configuración manual en el entorno local.
Stack tecnológico	El stack es completamente libre siempre que se cumplan los tres requisitos anteriores. Se valorará la justificación de las decisiones tecnológicas.

6. Ingeniería de Software Requerida

6.1. Levantamiento de Requerimientos

Se debe presentar un documento o sección de documentación que contenga:

- Requerimientos funcionales: qué debe hacer el sistema.
- Requerimientos no funcionales: rendimiento, seguridad, escalabilidad, usabilidad.
- Restricciones técnicas y de negocio.
- Supuestos y dependencias del sistema.

6.2. Casos de Uso

Se deben definir los actores del sistema y sus interacciones principales:

Actor	Responsabilidades
Administrador general	Gestiona configuración, usuarios, sucursales y tiene visibilidad total del sistema.
Gerente de sucursal	Supervisa operaciones de su sucursal, aprueba transferencias y consulta reportes.
Operador de inventario	Realiza ingresos, retiros, solicita transferencias y registra ventas/compras.
Sistema externo (opcional)	Puede integrarse con ERPs o sistemas de punto de venta existentes vía API.

6.3. Historias de Usuario (Recomendadas)

Si bien son opcionales, se recomienda documentar las funcionalidades clave en formato de historia de usuario, ya que facilita la evaluación y comunica claramente el valor de cada módulo:

Como operador de inventario, quiero registrar el ingreso de productos con su precio de compra, para mantener el costo promedio del inventario actualizado y generar órdenes de pago a proveedores.

Como gerente de sucursal, quiero ver en un dashboard la comparativa de ventas entre el mes actual y los tres meses anteriores, para identificar tendencias y tomar decisiones de compra anticipadas.

Como operador de inventario, quiero solicitar la transferencia de un producto desde otra sucursal con indicación de urgencia, para que la sucursal origen pueda priorizar el despacho según disponibilidad.

7. Modelado del Sistema

7.1. Diagramas Obligatorios

El candidato debe incluir en la documentación o como archivos separados los siguientes diagramas de ingeniería:

- Diagrama de casos de uso: actores y sus relaciones con los módulos del sistema.
- Diagrama de actividades o flujo: al menos el flujo de transferencia entre sucursales y el flujo de venta.
- Diagrama de arquitectura: vista técnica del sistema con capas, servicios y base de datos.
- Diagrama entidad-relación (E-R): modelo de datos completo con relaciones.

Se sugiere el uso de herramientas como draw.io, Lucidchart, PlantUML o Mermaid para la generación de diagramas. Incluirlos como imágenes en el README o como archivos separados en el repositorio.

8. Arquitectura y Diseño Técnico

8.1. Separación de Responsabilidades

El sistema debe implementar una separación clara entre las siguientes capas:

Capa	Descripción
Capa de presentación (Frontend)	Interfaz de usuario responsiva. Puede implementarse con React, Vue, Angular o cualquier framework moderno. Toda comunicación con el backend debe realizarse a través de la API definida.
Capa de negocio (Backend)	API RESTful o GraphQL que centraliza la lógica de negocio. Responsable de validaciones, reglas de transferencia, cálculo de costos y generación de reportes.
Capa de datos (Base de datos)	Almacenamiento persistente. Puede ser relacional (PostgreSQL, MySQL) o NoSQL (MongoDB), según la justificación técnica del candidato.
Infraestructura (Docker)	Toda la solución debe ejecutarse mediante Docker Compose, con servicios independientes y correctamente aislados para frontend, backend y base de datos.

8.2. Decisiones Técnicas a Documentar

Cada decisión de arquitectura significativa debe quedar documentada con su justificación. Como mínimo, se esperan justificaciones para:

- Elección del lenguaje de backend y sus ventajas para el contexto del problema.
- Selección del motor de base de datos y el modelo de datos adoptado.
- Estrategia de autenticación y autorización implementada.
- Mecanismo de sincronización de inventario entre sucursales.
- Cualquier patrón de diseño utilizado (Repository, Factory, CQRS, etc.).

9. Uso de Inteligencia Artificial en el Desarrollo

El desarrollo debe estar guiado y potenciado por herramientas de inteligencia artificial. Lejos de ser un simple apoyo, el uso de IA debe ser un componente estructural del proceso de desarrollo, y su aplicación debe quedar debidamente documentada y justificada.

9.1. Áreas de Aplicación Sugeridas

Área	Descripción	Impacto
------	-------------	---------

Diseño de arquitectura	Usar IA para evaluar opciones de arquitectura, comparar patrones de diseño y documentar decisiones con argumentos técnicos sólidos.	Alta
Generación de código	Uso de asistentes de código (GitHub Copilot, Claude, ChatGPT) para acelerar la implementación de módulos repetitivos como CRUDs, validaciones o endpoints.	Alta
Generación de tests	Creación automática de pruebas unitarias e integración para los módulos críticos del sistema.	Media
Documentación técnica	Redacción y estructuración del README, docstrings, comentarios de código y documentación de la API.	Alta
Revisión de código	Análisis de código generado para detectar vulnerabilidades, antipatrones o posibles optimizaciones.	Media
Consulta de buenas prácticas	Investigar estándares de industria, convenciones de nomenclatura y patrones recomendados para el stack elegido.	Media

9.2. Evidencia Esperada

En la documentación del proyecto se debe incluir una sección dedicada al uso de IA que contenga:

- Descripción de las herramientas de IA utilizadas y en qué etapa del desarrollo.
- Ejemplos concretos de prompts utilizados y resultados obtenidos (capturas o fragmentos).
- Evaluación crítica: ¿qué aportó la IA? ¿Qué fue necesario ajustar manualmente? ¿Dónde no fue útil?
- Estimación del porcentaje del código o documentación generado con asistencia de IA.

El uso de IA no es una señal de debilidad técnica, sino de madurez profesional. Lo que se evalúa es la capacidad del candidato para dirigir, validar y mejorar el output generado por estas herramientas.

10. Entregables Esperados

Entregable	Descripción
Repositorio GitHub	Repositorio público con el código fuente completo, historial de commits representativo del proceso y estructura de carpetas clara.
Código fuente	Código de frontend, backend y scripts de base de datos. Debe estar organizado, comentado y libre de archivos innecesarios (.env, node_modules, etc.).
Docker Compose	Archivo docker-compose.yml que levante toda la solución con un solo comando: docker compose up. Se incluyen instrucciones de configuración inicial.
Documentación	README completo con descripción del proyecto, instrucciones de instalación, arquitectura, módulos implementados y decisiones de diseño.
Diagramas de ingeniería	Al menos: diagrama de casos de uso, diagrama de actividades, diagrama de arquitectura y diagrama E-R en formato imagen o como archivos de herramienta de diagramado.
Sección de IA	Documento o sección dedicada al uso de inteligencia artificial durante el desarrollo, con evidencias y evaluación crítica.

12. Tiempo y Consideraciones Finales

Para aprovechar al máximo el tiempo disponible, se sugiere el siguiente orden de trabajo:

- Definir arquitectura y stack tecnológico con justificación documentada.
- Modelar la base de datos y crear los diagramas de ingeniería iniciales.
- Configurar el entorno Docker y estructura base del proyecto.
- Implementar el backend con los módulos en orden de dependencia.
- Desarrollar el frontend conectado a la API.
- Implementar la funcionalidad adicional propuesta.
- Redactar documentación, revisar con IA y refinar entregables.

*Cada decisión de diseño debe poder responder: '**¿Por qué se hizo así?**' — Esta es la pregunta central que guía la evaluación del proyecto.*